

Can compressed attention compete under equal GPU time?

DeepSeek-style Compressed Sparse Attention, small-scale repo pilot

Headline

Under equal GPU time, dense attention clearly beat this plain-PyTorch CSA implementation. CSA looked competitive only in the less fair fixed-token view.

What this report is

A compact research artifact for teaching: one honest result, the fairness rule, the table, the chart, the caveats, and the next clean experiment.

METADATA

Field	Value
Date	2026-05-24
Hardware	1x NVIDIA RTX 3090, 24GB VRAM
Evidence run commit	b2a40c1a6830
Primary evidence folder	docs/research/results/csa_top_k_20260524

RESEARCH QUESTION

If CSA can read compressed summaries of old tokens, does increasing top_k recover quality under a fixed GPU-time budget?

Fairness rule

Same model, data, batch size, sequence length, optimizer, seed, hardware, and active training-time budget. VRAM is measured as an outcome, not reused to increase batch size.

CONDITIONS

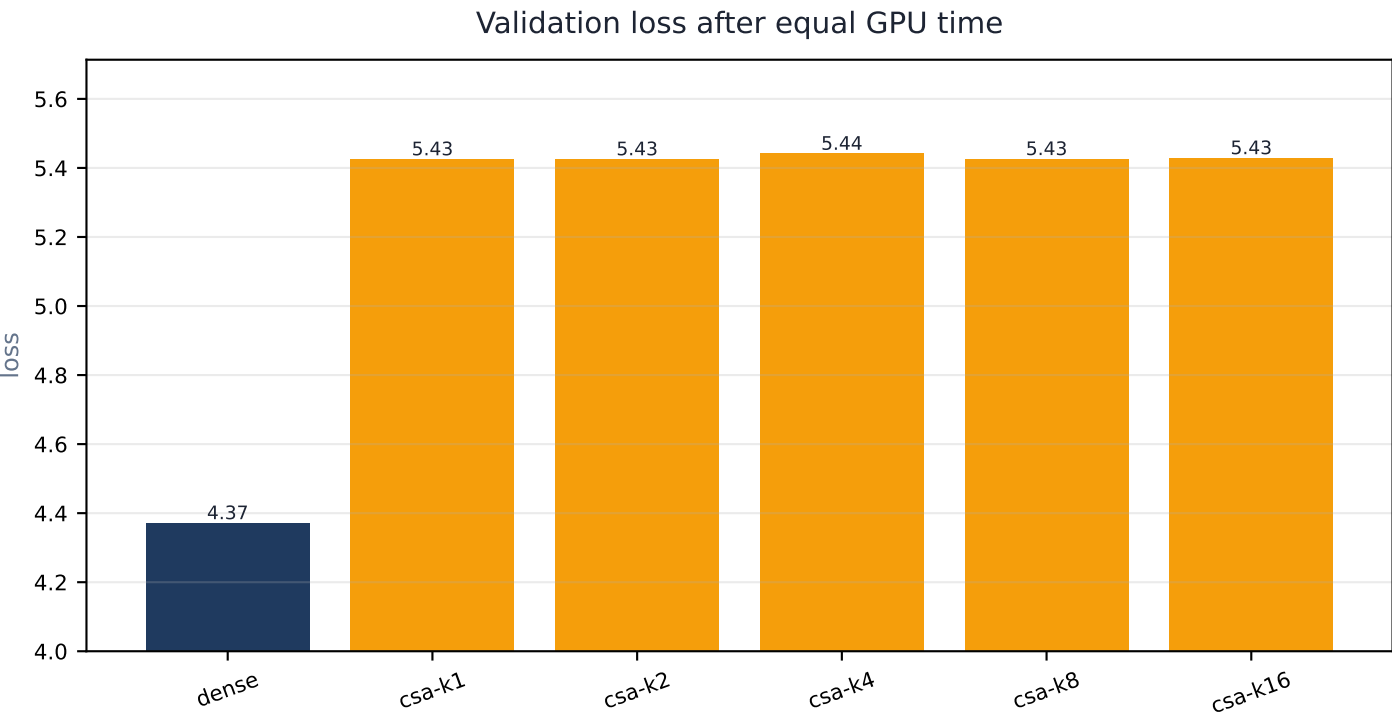
Run	Reads	Why included
dense	Full causal past	strong baseline
csa-k1	Local window + 1 compressed block	small compressed memory
csa-k2	Local window + 2 compressed blocks	slightly more memory
csa-k4	Local window + 4 compressed blocks	default pilot knob
csa-k8	Local window + 8 compressed blocks	larger memory
csa-k16	Local window + 16 compressed blocks	largest memory in sweep

IMPORTANT CAVEAT

This is a plain-PyTorch research implementation. Dense attention benefits from optimized PyTorch kernels. Therefore the result is about this implementation and setup, not a universal claim that CSA cannot work.

FIXED GPU-TIME RESULT

All rows used about 300 active training seconds. This is the headline comparison because each setting receives the same wall-clock compute budget.



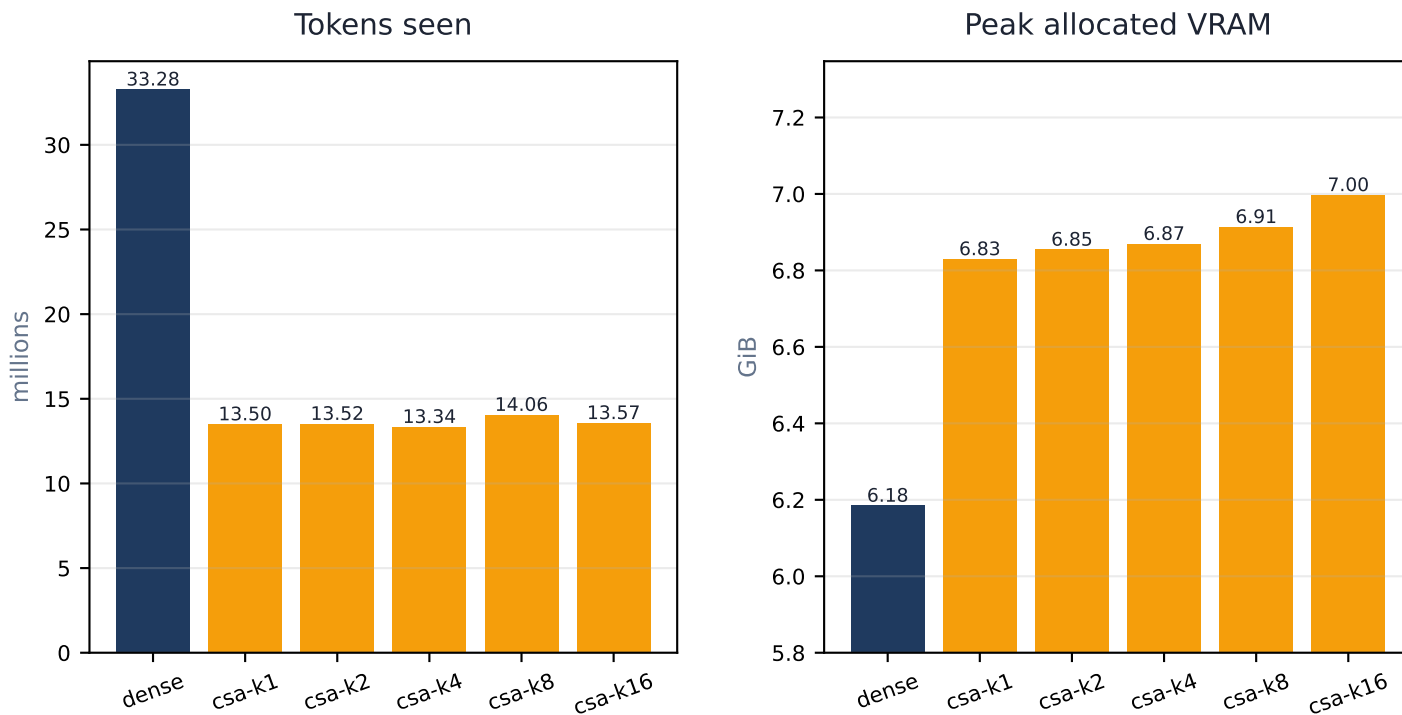
Run	Val loss	Tokens seen	Tok/s	VRAM GiB	Coverage
dense	4.3713	33,275,904	109930	6.18	2048
csa-k1	5.4255	13,500,416	44373	6.83	80
csa-k2	5.4255	13,524,992	44442	6.85	96
csa-k4	5.4414	13,344,768	43848	6.87	128
csa-k8	5.4261	14,057,472	46198	6.91	192
csa-k16	5.4287	13,574,144	44627	7.00	320

Readout

Dense reached 4.3713 validation loss while the CSA rows clustered near 5.43. Increasing top_k did not produce a visible quality recovery.

THROUGHPUT AND MEMORY

The fixed-time result is partly explained by tokens processed. Dense trained on more than twice as many tokens in the same active training window.



WHAT THE BUDGET SAYS

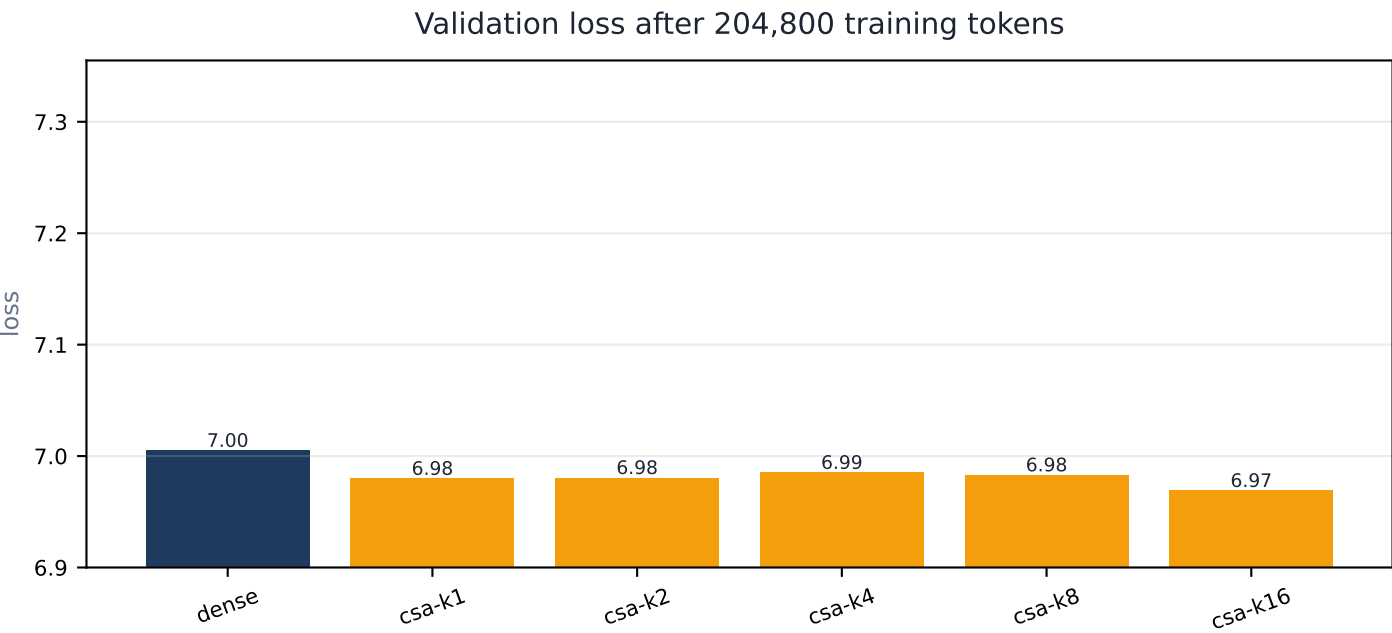
- Dense processed 33.3M tokens in about 300 seconds.
- CSA rows processed about 13.3M to 14.1M tokens in the same time.
- CSA allocated slightly more VRAM in this implementation.
- Theoretical sparse attention savings did not appear as a practical speed win here.

Why this matters

A same-token comparison can make CSA look competitive, but equal GPU time asks the practical question: what do we get for the same compute budget?

FIXED-TOKEN DIAGNOSTIC

The fixed-token view is useful as a diagnostic, but it is not the headline fairness view because different methods can spend different compute per token.



NEXT CLEAN EXPERIMENT

For the teaching video, the simpler experiment is dense vs local-only vs CSA. It asks whether compressed memory helps beyond a local attention window.

Condition	Interpretation
dense	read the full causal past
local	read nearby tokens only
csa	read nearby tokens plus compressed older memory

Recommended run

3 attention settings x 3 seeds x 10 minutes = about 90 GPU-minutes. If the gap is below 0.03 validation loss, do not claim a winner.